STEP File Classification and Annotation using Transformer Architecture

UNDERGRADUATE FINAL YEAR PROJECT

Hans Sebastian
2022390007

Computer Science
Faculty of Engineering annd Technology
Sampoerna University
Jakarta
2025

STEP File Classification and Annotation using Transformer Architecture

UNDERGRADUATE FINAL YEAR PROJECT

Hans Sebastian
2022390007

Supervisor 1:	**Supervisor 2:**
Panji Darma. Ph.D	**Supervisor 2 Name with title**

December 14, 2025

DECLARATION OF ORIGINALITY

I, Hans Sebastian 2022390007, here acknowledge that this submission entitled "STEP File Classification and Annotation using Transformer Architecture" is my own work with the guidance from my supervisor. I also truly acknowledge that it contains no materials previously published or written by another person, or any other educational institution, except where due acknowledgment is made in the Final Year Project. Any contribution made to the research by others is explicitly acknowledged in the Final Year Project.

Jakarta, December 14, 2025

Hans Sebastian

EXCLUSIVE RIGHT STATEMENT

I, Hans Sebastian 2022390007, hereby grant to our school, Sampoerna University, the non exclusive right to archive, reproduce, and distribute my Final Year Project entitled “STEP File Classification and Annotation using Transformer Architecture” in whole or in parts, whether in printed or electronic formats. I acknowledge that I retain exclusive rights of my Final Year Project by using all or parts of it in future work or output, such as articles, books, software, and information system.

Jakarta, December 14, 2025

Hans Sebastian

ACKNOWLEDGEMENT

Thanks to

Jakarta, December 14, 2025

Hans Sebastian

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABSTRACT

Full Name : Hans Sebastian

Study Program : Computer Science

Title : STEP File Classification and Annotation using Transformer Architecture

Supervisor : Panji Darma. Ph.D

The management of Computer-Aided Design (CAD) assets is currently hindered by a reliance on manual annotation and the limitations of existing automated classification methods. Current approaches often require lossy conversion to visual representations or depend on synthetic construction histories unavailable in industrial Boundary Representation (B-Rep) files. This study proposes a novel Transformer-based architecture for the direct classification and annotation of raw STEP files. Central to this methodology is a Hierarchical Dual-Stream Tokenizer designed to process non-Euclidean B-Rep graphs. The pipeline implements Semantic Macro-Compression to reduce graph complexity and utilizes Canonical Normalization via Principal Component Analysis (PCA) to ensure translational and rotational invariance. To resolve graph isomorphism ambiguity, the system applies a deterministic Topological Linearization using a hierarchical sorting key. The resulting model utilizes a Dual-Stream Encoding strategy, fusing discrete semantic tokens with complex-valued geometric embeddings through a Geometry-Infused Attention mechanism and Rotary Positional Embeddings (RoPE). Validated against a hybrid dataset constructed from Text2CAD, ABC, and TraceParts using a Human-in-the-Loop pipeline, this approach effectively captures high-level functional identity without reconstructing procedural history.

Keywords: ...

CHAPTER I

INTRODUCTION

1.1 Background

1.1.1 Blueprinting and Other Traditional Copying Methods

Before digital technology became widespread, engineers, architects and designers used conventional methods such as mylars and blueprints which require the drawings and measurements to be copied manually by dozens of workers. Blueprints utilized chemicals such as potassium ferricyaniide and ferric ammonium citrate solution to coat the original drawing to be copied. The coated paper is then exposed to ultraviolet light, creating a white-on-blue copy after washing it with water (Herschel, 1842). This method suffers from 2 main drawbacks which limits productivity and flexibility when designing. One such drawback is the modification issue, which is that the master design to be fixed before copying, meaning any changes to the original causes the copies to be deprecated and the entire process redone from scratch. The other issue comes with the need to dry the copies before they become usable, creating more overhead.

Whiteprinting attempted to replace the original blueprinting technique by utilizing ammonia fumes instead of water, resulting in a black-on-white copy that does not need to be dried (Parmeggiani, 2014). Although it improved efficiency by creating a dry copy, whiteprinting includes the creation of hazardous fumes such as ammonia which could cause long-term damage to the operator (McGlothlin, 1981). Furthermore, this technique still suffer from the modification issue that plagued the original. These downsides pushed engineers to develop better, more efficient alternatives.

1.1.2 Computer Aided Design

Advances in technology have allowed for the development of better procedures, especially computer aided design (CAD) which largely alleviated the issues present in prior methods. CAD can be defined as the use of digital computers to create, modify, analyze, and optimize existing designs (Sadrehaghighi, 2022). In other words, CAD is the process of utilizing digital technology to perform the entire design process, from ideation to generation and sharing, bypassing the need for traditional copying procedures such as blueprinting. The concept originated from the first program *Sketchpad* designed by Ivan Sutherland in 1963 (Sutherland, 1964), and has received numerous improvements with newer tools and programs such as SolidWorks (Zeid, 2021). A short diagram detailing the CAD process can be seen in Figure 1.1.

In the current era, the use of CAD tools and software have largely

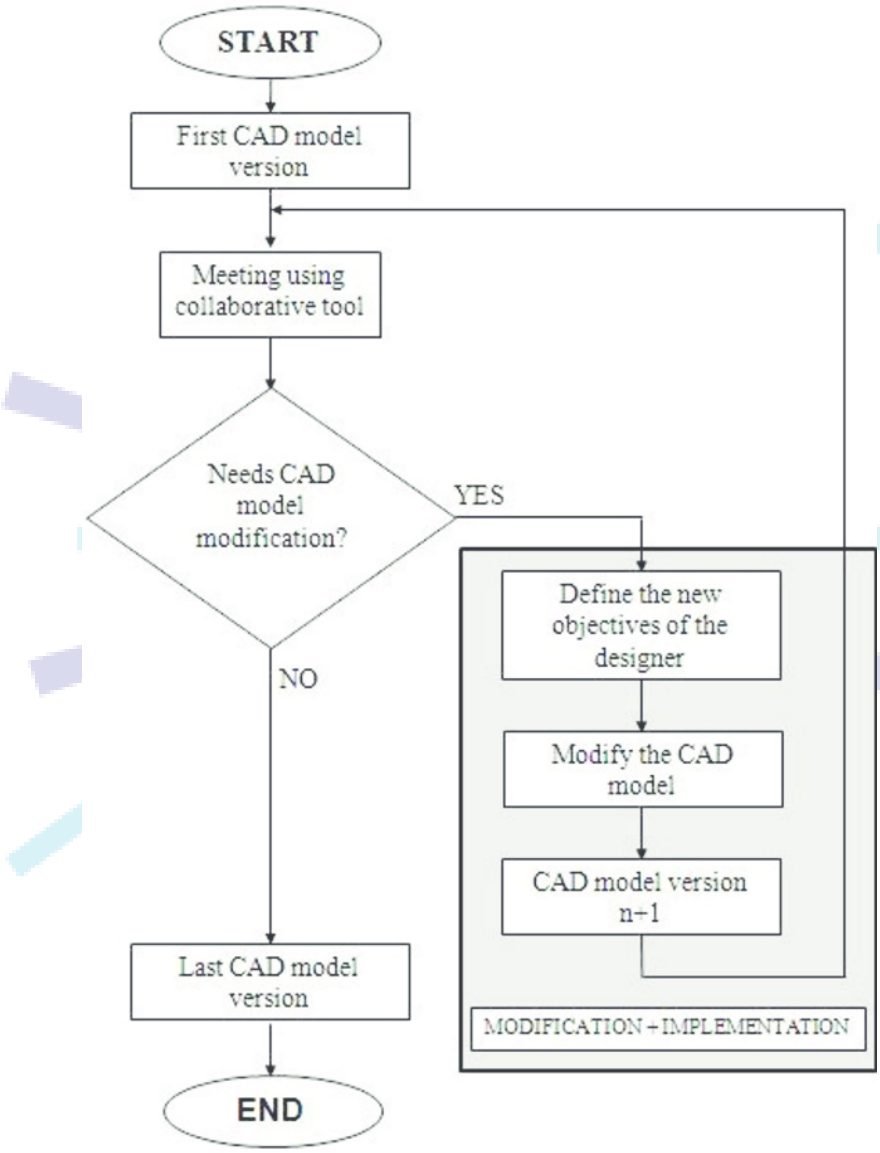

Figure 1.1: Flowchart of the CAD modeling process (Fleche et al., 2014).

replaced such methods as they lack the limitations that come with traditional techniques. This is especially true in engineering-related fields with one study showing up to 50% reduction in task duration (Rahman et al., 2019; Gutiérrez de Ravé et al., 2025). Other benefits include the ability for immediate changes even during late-stage development which helps limit delays (Aranburu et al., 2021). The main benefit of CAD tools is ability to reutilize/repurpose existing CAD designs to be reused in other CAD projects, resulting in more efficient, reliable workflows that eliminates the need to redesign already existing components from scratch (Gutiérrez de Ravé et al., 2025).

Although the use of CAD have improved the overall design and man-

ufacturing process, they also pose their own set of issues. For example, CAD software developers often employ their own proprietary file formats, each with their own complex structures. This results in major incompatibility issues as users might have different tooling preferences. As a result, international governing bodies such as International Organization for Standardization (ISO) have developed universal CAD file formats such as STEP to handle the issue of compatibility. A STEP file consists of components that each describe geometric data using a language called EXPRESS (Pratt, 2001). Another universal CAD file format include STL designed by 3D Systems in 1987, which contains a set of triangles consisting of facets and vertices (Szilvsi-Nagy and Mátyási, 2003). Another problem comes with difficulties in documentation and indexing for existing CAD designs, as the complexity of the file's structure inhibit the use of automated annotation and indexing tools, forcing the use of manual annotation. Although the universal file formats help with compatibility between different CAD software, issues regarding documentation and indexing still persists, prompting the research for potential solutions.

1.2 Problem Formulation

There are multiple issues relating to CAD design annotation, one example being that manual annotation incurs an unnecessary overhead, diverting manpower and time away from the main design process. Furthermore, manual annotation also introduce risks regarding annotation quality and accuracy, as human annotators might miss important details or mislabel important components, reducing overall workflow efficiency (Mandelli and Berretti, 2022). Difficulties in determining annotation formats and standards increases risk of errors, which is especially noticeable when dealing with third-party designs.

The development of artificial intelligence (AI) have resulted in new techniques for automating the process, however most AI-based methods and models require the use of additional conversion steps into images and/or point clouds to leverage existing image processing and computer vision techniques (Mandelli and Berretti, 2022). This added steps reduce the overall efficiency and increases the risk of misidentification/misclassification. This study aims to develop a novel method for processing CAD files which preserves the existing semantic context and connections between the parts without the need for additional conversion steps into visual representations. Additionally, current state-of-the-art (SotA) models such as Gemini 3 and Cha:t-GPT 5.1 overly rely on the existing metadata contained in the file, completely bypassing the geometry of the design. Other issues related to these models include privacy and latency concerns due to the necessity for an internet connection, as well as high computational and energy costs as the models are designed for general use (O'Donnell and Crownhart, 2025).

1.3 Research Objectives

Based on the topic discussed in the Problem Statement section, this study aims to complete the following research objectives:

1. Develop a Transformer-based system capable of generating high-fidelity functional descriptions for unlabelled STEP files, validated against human-authored ground truth using BERTScore.
2. Translating low-level B-Rep topology (e.g., NURBS curves, closed shells) into high-level semantic concepts (e.g., "Hexagonal Head", "Threaded Shaft") with $> 80\%$ identification accuracy.
3. Enhance CAD file indexing by leveraging model-generated descriptions as semantic metadata.
4. Enable low latency inference using a Hierarchical Dual-Stream Tokenizer architecture. (At least < 1s inference time for any given input model).

1.4 Significance of Study

This study's significance and contributions towards existing literature are detailed as follows:

1. **Direct B-Rep Processing:** Eliminates the need for lossy conversion to 2D images or voxels by processing raw STEP files directly, preserving high-fidelity NURBS geometry and topological relationships (Mandelli and Berretti, 2022).
2. **History-Agnostic Classification:** Overcomes the reliance on synthetic construction history found in existing datasets (e.g., Text2CAD) by extracting functional identity from static B-Rep graphs, making the model applicable to standard industry files (Khan et al., 2024).
3. **Novel Graph Serialization:** Introduces a **Hierarchical Dual-Stream Tokenizer** that resolves graph isomorphism ambiguity in non-Euclidean data, ensuring deterministic processing of scrambled geometric entities (Wu et al., 2021).

1.5 Limitation of Study

The following study lists the following limitations and potential improvements:

1. **Context-dependence:** The labeling of CAD objects relies on designer intent, which might be lost during pre-processing.
2. **Implicit Bias:** The correctness and quality of the output and training process are determined based on human responses, which introduces potential bias.

3. **Limited Vocabulary:** The model's output and training data are based solely on the English language, limiting its ability to generate annotations in different languages.
   Additionally, the model's vocabulary is modified to reduce subjectivity by limiting the use of adjectives such as "big" or "small".
4. **Sparse classes:** The model's training data only consists of general common objects which can be easily found in public datasets in order to maximize the accuracy of the model. This is due to the scarcity of complex CAD designs in such datasets as they contain mostly reusable assets.

1.6 Organization of the report

The rest of the paper will be organized according to the following structure: Chapter II analyses the existing literature and basic terminologies required to understand the topic. Chapter III details the methodology used to gather the training dataset and the design considerations related to model development, from tokenizing, fine-tuning, to the benchmarking process. Chapter IV lists the results gathered during the benchmarking process paired with detailed analysis of said results. Lastly, Chapter V summarizes the conclusion obtained from the study as well as potential improvements for future research.

CHAPTER II

LITERATURE REVIEW

2.1 Data and Information

2.1.1 Definitions

Data refers to a collection of raw, unlabelled information which represents a set of truth/facts about the world (Olson, 2021). Data comes in two main categories, quantitative and qualitative (Hackman et al., 2024). Quantitative data refers to data which has a distinct form, with unique categories and can be analyzed using objective numerical operations and methods, where qualitative data lacks a concrete structure and is highly context-dependent and subjective (Schoonenboom, 2023). Quantitative data mostly consists of values that can be represented numerically (a person's height/weight, price of an object), while qualitative data refers to data about a subject such as nominal descriptions (a person's name, a novel's synopsis). This paper focuses on digital data, which is data represented as a sequence of symbols, mainly binary digits.

The concept of data differs from information in that data serves as the foundation from which information is generated (Hackman et al., 2024). In essence, information is a specific interpretation of a given dataset, derived from their type and values, as well as their context and semantic meaning. In particular, the context in which a collection of data is acquired determines the set of valid interpretations of said data, which in turn affects the possible inferences i.e. potential information gathered from it (Mason, 2019). Raw data in itself cannot be used directly, as it merely represents the attributes of a certain subject, without assigning to it any valid implication/insight. To make use of data, it needs to be converted into information through analysis and contextualization (Borrohou et al., 2025).

As the name suggests, information is the basis of all modern information technologies and systems such as search engines, recommendation systems, and artificial intelligence. These systems utilize information gathered from large datasets to gain knowledge, derive solutions and make decisions such as giving a custom-tailored recommendation to particular set of users in the case of recommendation systems, and generating responses in the case of AI. The amount of data generated daily has grown exponentially in the information age, with one estimate stating approximately 402.74 million terabytes are generated by electronic devices around the world daily, and an estimated 394 zettabytes will be generated by 2028 (Taylor, 2025). This prompts the need to develop a method for organizing, analyzing, and managing large batches of data efficiently and accurately.

2.1.2 Data classification

Data classification is the process of assigning labels to data for better management purposes (Newhouse et al., 2023; Shivayogi, 2025). In short, data classification is the process of grouping data into different categories according to their characteristics for later processing and bookkeeping. Data classification allows for enhanced protection, security and risk management of data based on sensitivity and regulatory standards (Lane and Adelusi, 2023; Nguyen and Cuong, 2025; Newhouse et al., 2023). Furthermore, data classification allows for better organization and retrieval of certain types of data, improving the accuracy and performance of information gathering. Data classification methods largely follows this specific sequence of steps: 1) defining classification criteria, 2) data discovery and sensitive data detection and 3) data labeling based on type, usage, and security requirements (Shivayogi, 2025).

Most forms of data, such as text, images, and video/audio recordings, are categorized as qualitative as they lack a formal, fixed set of attributes and structure. Furthermore, data with complex structures such as CAD files contain a large amount of features and attributes that form deep semantic relationships. As a result, these types of data cannot be easily classified using conventional methods as they rely on assumptions regarding the data's structure. Recent advancements in artificial intelligence systems such as natural language processing and deep learning have allowed for reliable, efficient classification of these datatypes (Blessing, 2021). In modern systems, vector embeddings have become the cornerstone that enables these capabilities, replacing the less efficient rule-based approach in natural language processing, and acting as the basis for deep learning systems such as convolutional/recurrent neural network models (Patil et al., 2023; Asudani et al., 2023). This paper will focus on the use of vector embeddings as a method for enumerating and processing CAD files for classification purposes.

2.2 Vector Embeddings

2.2.1 Definition and Purpose

Vector embedding is defined as a method for converting and representing complex, unstructured data as a sequence of numerical values (Pajo, 2025; Chitturi, 2024). A vector embedding aims to create a compact, structured representation of an underlying datapoint while preserving the underlying context and semantic meanings contained in it (Chitturi, 2024). In other words, vector embeddings encode unstructured, context-rich data into points in an n-dimensional space, capturing the relationships of different data points in numeric form, allowing the data to be processed and analyzed using mathemathical operations (Pajo, 2025). This representation also renders the concept of "distance" as a quantifi-

able component in data analysis, where distance refers to semantic proximity/closeness in meaning (Kukreja et al., 2023). In short, the distance between n-dimensional vector representation of two datapoints closely models the semantic relationship between the original data.

Vector embedding serves as a way to efficiently encode large datapoints such as texts, images, video and audio into a digestible, compact format that could be easily processed by other systems. Vector embeddings also allows data to be searched, indexed, and grouped using their semantic similarity (Taipalus, 2024). One use case is in large language models, where models could be equipped with the ability to access external documents and data, allowing it to produce reliable output grounded on falsifiable truth (Omolayo and Babatope, 2025).

Other similar methods for encoding qualitative, complex data have been developed, such as one-hot encoding, which encodes each datapoint as states represented as a sequence of n-bits, where all but one bit is set to '0', and the '1' bit represents the current state of the system (Samuels, 2024). Another example is Bag of Words (BoW), which represents text as a "bag"; an unordered, unstructured collection of its words, or keypoints in the case of an image (Qader et al., 2019), and Term Frequency-Inverse Document Frequency (TF-IDF), which encodes words/keypoints as its number of occurrence and rarity inside a document (Qaiser and Ali, 2018).

Nonetheless, the methods listed previously face several limitations when compared to newer techniques such as vector embedding, such as the loss of semantic information such as the word order, grammatical structure, and other relevant context stored in the original. Other problems include the problem of dimensionality, which states that as the number of dimensions increase for a given set of data, the amount of available data becomes increasingly sparse and patterns more obscure (Banks and Fienberg, 2003). This limitation also massively increases the compute power and memory required to store and process these encodings while also increasing the risk of errors (Trunk, 1979). Vector embedding sidesteps these limitations by mapping similar items to vectors which have a close distance to each other, preserving semantic data and minimizing the number of features required. The shift from sparse, count-based encoding methods towards a more compact, learned encoding technique enables critical performance improvements in terms of data processing metrics such as compute time and output accuracy.

2.2.2 Embedding Models

Although vector embedding offers several important advantages, such advantages require specialized, novel machine learning models that are

capable of distilling complex relationships between datapoints into a compact dense vector representation while minimizing the amount of features/dimensions. These models capture context ingrained within datapoints by learning how to assign a connection between two neighboring words/parts of data (Tran, 2024, Chapter 4.1).

Most embedding models could be categorized into two groups: static models that generate a single, fixed representation for a word/data segment, such as continuous BoW and Subword models (Wang et al., 2019), and contextual models such as Bidirectional Encoder Representations from Transformers (BERT) and its variants that generate unique representations of the same word depending on the current context (Liu et al., 2025; Devlin et al., 2019). In recent times, contextual models have largely dominated due to their flexibility and better capabilities in terms of capturing the nuance and implicit interconnections between datapoints and its components. One popular example of this is the transformer architecture, which forms the basis the paper's research.

2.3 Transformer Architecture

2.3.1 Architecture Overview

Current state-of-the-art (SotA) embedding models are based on the transformer architecture, which acts as the foundation for commercial generative AI models such as ChatGPT, Gemini, DeepSeek, etc. Introduced in the seminal paper "Attention Is All You Need" by Vaswani (Vaswani et al., 2017), instead of processing data sequentially, the architecture utilized a novel approach for sequence-processing tasks referred in the paper as self-attention, replacing the recurrent, in-order structures of previous models like recurrent neural networks (RNNs) and long short-term memory models (LSTMs) (Sherstinsky, 2020; Hochreiter and Schmidhuber, 1997).

The models share their similarity in their use of tokenization, a process of dividing input data into smaller chunks referred to as tokens. The tokens are represented as numeric vectors, reducing the amount of storage and compute needed for further processing. The literature dictates two main paradigms of tokenizing input data, full word and subword tokenization. Full word tokenization, as it implies, splits input data based on a pre-defined separator, such as whitespace in text. The above method have largely been replaced by its counterpart, which divides input data into smaller subparts that has no fixed separator. Examples of subword tokenization include byte pair encoding and WordPiece (Suyunu et al., 2024; Kozma and Voderholzer, 2024; Devlin et al., 2019). The tokens generated from the process are then used as the main input data of the self-attention mechanism.

The self-attention mechanism detailed in the transformer architecture improves upon previous systems by enabling models to evaluate the importance of the surrounding parts of an input sequence, regardless of its ordering. For instance, consider the sentence "The dog barked at the cat to drive it away." Using self-attention, transformer-based models can correctly associate the word "it" with the word "cat" rather than "dog" (Vaswani et al., 2017). This capability of handling remote dependencies and understand context allows it to outperform previous architectures. The details of the mechanism lies in the novel components that make up the architecture, forming a deep network. In particular, the transformer model's functionality and reliability is the outcome from combining two key components, encoders and decoders. Figure 2.1 details the architecture of a standard transformer.

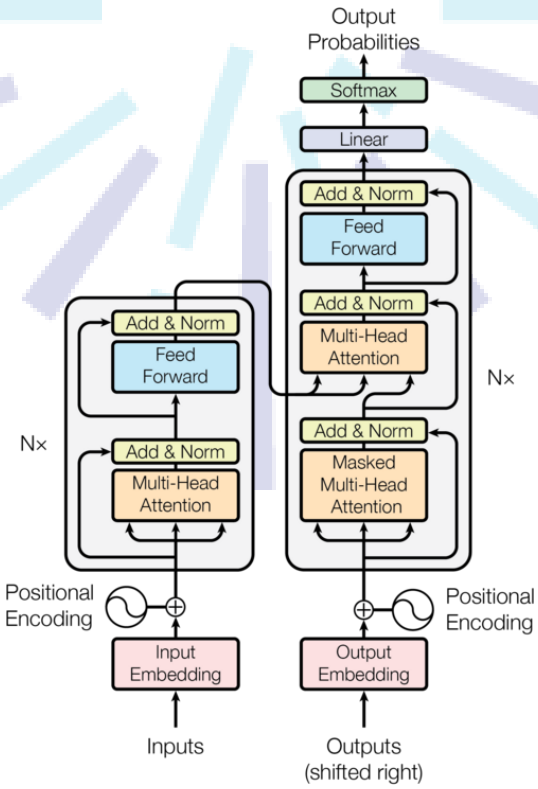

Figure 2.1: Standard Transformer Architecture consisting of an Encoder and Decoder (Vaswani et al., 2017)

2.3.2 Encoder-Decoder

The encoder-decoder components consists of several blocks/subcomponents that in turn form the basis of a transformer layer, which are then stacked continuously to build the complex models. The encoder functions as a processing layer for the entire input sequence, aiming to build a rich, contextualized numerical representation utilizing the self-attention mechanism. The encoder is implemented as two main parts:

1. **Multi-Head Self-Attention:**
   The self-attention of the transformer is employed as different units/heads working in parallel, allowing the model to learn different types of relationships from the input data simultaneously (Voita et al., 2019). Each "head" is tasked with learning a different aspect of the given sequence. For instance, one head might focus on syntactic relationships while another learns semantic ones. This functionality is done through projecting the input data into different representation subspaces and applying self-attention in parallel. The final result is a vector generated by applying a linear transform to the concatenation of the heads' output (Vaswani et al., 2017)
2. **Position-wise Feed-Forward Networks (FFN):**
   The next step consists of passing the token vectors gathered and weighted from the attention mechanism through a feed-forward neural network. The subcomponent consists of two linear neural network layer with a non-linear activation function in between, such as ReLU. It enriches each token's representation by first expanding the vector's dimension (i.e. 512 to 2048), applying an activation layer and resizing the vectors back into its original size. This process allows the model to extract deep, interconnected patterns hidden within the data. Furthermore, the network acts as a key-value store for patterns learned during training, and its output represents a distribution of tokens containing said patterns (Geva et al., 2021).

The parallel nature of self-attention makes it difficult to retain information regarding sequence ordering. Hence, positional encodings in the form of special vectors are added to the embedddings to mitigate this limitation. The positional encoding vectors are generated using sinusoidal functions with different frequencies that represent positions of tokens inside the sequence (Vaswani et al., 2017). After the encoding process, the result generated by the encoder is passed through a decoder that remaps the resulting vectors back into tokens that forms the final output sequence. The decoder achieves this through three steps:

1. **Masked Multi-Head Self-Attention:**
   The first step of the decoding sequence is similar with encoding, with a key difference being he lack of a lookahead for future tokens. This is done to prevent model overfitting as knowing future tokens might embed irrelevant connections into the model.
2. **Cross-attention:**
   Another change made in the decoding process is the addition of a cross-attention layer. The layer acts as a critical link between the encoder and decoder that functions as a window for the decoder to look at the encoder's final output. The encoder uses its internal state as a query in the form of a vector and relates it to the vector set generated by the encoder. This process enabels the decoder to determine the most relevant parts of the input and its relation to the final

output (Vaswani et al., 2017).

3. **Position-wise Feed-Forward Networks (FFN):**
   The final decoding step is similar to the encoding process with no changes.

Enabling effective training of densely-layered networks require proper integration of the encoder and decoder. To meet these requirements, two operations are applied post-encoding and decoding to ensure stability and effective training. The integration involves adding a residual connection layer that adds the original input vector with the output to minimize the vanishing gradient problem. Next, the vectors are normalized and stabilized, ensuring a smoother and reliable training process. The architectural decisions that make up the model allows the connections between each token to be filtered and strengthened based on the input's context, and allows the model to retain more information from its training data (Vaswani et al., 2017).

2.4 Related Works

Early attempts at classifying CAD files mainly used feature descriptors such as Zernike and Reeb graph descriptors to determine the category of a CAD model based on shape and function (Ip and Regli, 2005).

Several other works focuses on 2D based methods such as utilizing machine learning to classify over 1600 CAD files using random forest, support vector and fully-connected neural network models by identifying 45 different basic shapes (Machalica and Matyjewski, 2019), with subsequent approaches combining GNN to measure and group CAD files based on their similarities in geometry (Roj et al., 2021). More examples of this type include RotationNet, which utilizes images of the same 3D objects from different angles to estimate its pose and category (Kanezaki et al., 2018).

Different category of related studies uses a 3D-based approach, with one leveraging the use of 3D grid representation (voxels) of the CAD file and applying a 3-dimensional-CNN that could detect and categorize features (Maturana and Scherer, 2015), and another applies a Mask-recurrent CNN model to extract 3D objects from a single 2D RGB image which can then be classified (Gümeli et al., 2022). Works such as PointNet utilizes 3D point clouds to estimate the class of an object (Qi et al., 2016). A different classification uses a custom 3D representation named PANORAMA and feeding the result to a CNN to find keypoints and classify the CAD based on the detected features (Papadakis et al., 2009; Sfikas, Theoharis, et al., 2017; Sfikas, Pratikakis, et al., 2018). Newer 3D-based approaches utilizes graph-based techniques to make use of the graph-like structure of CAD files such as STEP, with one work using Graph-based Neural Network (GNN) to categorize certain CAD STEP

files into 7 different classes (Mandelli and Berretti, 2022). However, a research gap in previous works can be identified, since the cited works are limited to classifying a fixed class of CAD objects, such as bolts, and cannot be used to create a general description of objects not specified in the class dataset.

Another method applies a Feature Directed Acyclic Graph (FDAG) to determine the dependency between the shapes of CAD models which are then optimized until only the fundamental features and shapes remain. The shapes can then be compared to CAD models to determine their resemblance to the shapes used as a query (M. Li et al., 2011). One graph based method also utilizes the structure of STEP files to generate an attribute graph which can then be used as a measure for shape similarity (El-Mehalawi and Allen Miller, 2003a; El-Mehalawi and Allen Miller, 2003b). However, while these graph-theoretic approaches excel at topological retrieval, they remain disconnected from the semantic intent of the design.

To bridge the gap between geometric topology and natural language, recent frameworks have turned to multi-modal learning. Text2CAD provides a seminal contribution by pairing natural language with CAD geometry, its methodology relies entirely on Programmatic Synthesis (Khan et al., 2024). The dataset is constructed by first generating CAD scripts and then using Large Language Models (LLMs) to reverse-engineer textual descriptions from the code. Nevertheless, the approach introduces a research gap as real-world industry STEP files (ISO 10303-21) rarely contain the clean construction history (e.g., Extrude, Revolve) found in Text2CAD. Figure 2.2 details the overall flow of Text2CAD. They are often fixed Boundary Representations (B-Reps) containing only explicit surfaces and topology. Therefore, a model trained solely on Text2CAD fails to interpret the vast majority of preexisting CAD files. Moreover, the research focuses on generating the step-by-step geometric operations required for reconstruction (e.g. "Extrude by 2cm"). While this results in precise parametric descriptions, the model effectively describes the object's recipe without understanding what the object is. For instance, while it may generate highly accurate construction steps for a set of interlocking toothed cylinders and shafts by specifying exact relative measurements for every extrude and cut, it frequently fails to recognize or categorize the resulting assembly by its fundamental functional identity: a Gearbox. Consequently, the model provides the fabrication steps but lacks the capability to semantically categorize said object.

To overcome the dependency on synthetic construction history and the lack of functional understanding, this study introduces a tokenizer and annotation pipeline designed explicitly for Reconstruction-Free B-Rep Graphs. Furthermore, to ensure the model captures the high-level

semantic identity of components rather than just their geometric recipe, this research employs a Human-in-the-Loop (HITL) protocol to directly author and verify the dataset. This verification process guarantees that the descriptions align with the visual reality of the part, prioritizing functional categorization over procedural fabrication.

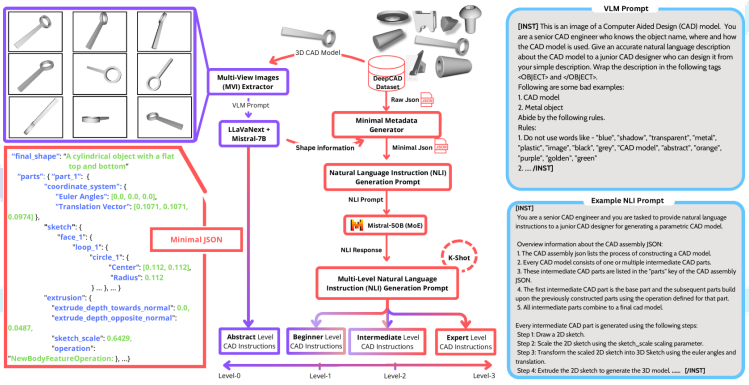

Figure 2.2: Text2CAD Annotation Generation Pipeline (Khan et al., 2024).

=

CHAPTER III

METHODOLOGY

3.1 Data Gathering

Training a descriptive model effectively necessitates a large-scale, high-fidelity dataset that pairs geometric representations with semantic natural language. However, unlike 2D image-captioning domains, the 3D CAD landscape suffers from a significant scarcity of standardized, human-readable annotations. To address this sparsity, a comprehensive acquisition framework was developed to ingest raw geometric data, filter for quality, and semantically enrich samples through a hybrid HITL process. A summary of the dataset preprocessing steps can be seen in Figure 3.1 below.

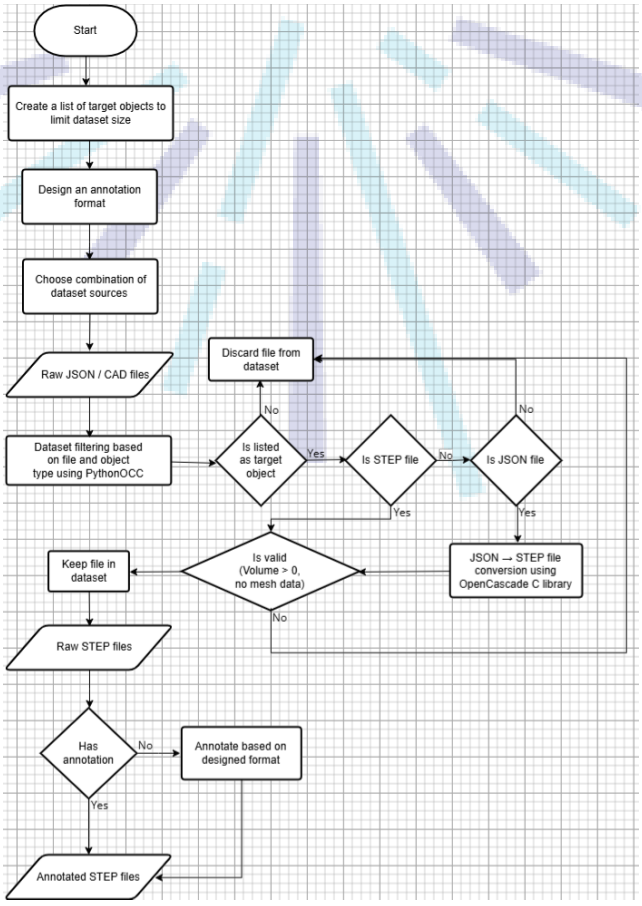

Figure 3.1: Dataset acquisition and filtering flowchart.

3.1.1 Dataset Source Selection

Publicly available datasets for CAD file designs can be found on the internet. One prominent example is the ABC dataset, which consists of over one million unannotated STEP files derived from Onshape (S. Koch et al., 2019). Another is the Fusion 360 Gallery, a dataset of free user-submitted

designs with design timeline (Karl et al., 2021), and Text2CAD, which hosts around 170,000 CAD models with 660,000 natural language annotations ranging from short descriptions to detailed explanations (Khan et al., 2024). Other sources include free 3D/CAD design websites such as GrabCAD and TraceParts, which contain various CAD objects not found in compiled datasets. A comparative summary for the sources listed above is contained in Table 3.1 below.

Source	Type	Summary	Advantages	Disadvantages
TraceParts	Online Library	Mfr-verified standard parts (bolts, gears, bearings).	High-fidelity geometry; ISO/DIN compliant.	Limited to standard hardware; requires scraping.
GrabCAD	Online Library	6M+ user files	High diversity of complex/creative designs.	Inconsistent quality; non-manifold geometry.
Text2CAD	Dataset	Annotated CAD files with text-geometry pairs.	Ready-to-use labels; strictly organized.	Lower complexity than industry files.
Fusion 360	Dataset	Designs exported with assembly timelines.	Includes reconstruction history (timelines).	Proprietary format; inconsistent exports.
ABC	Dataset	1M+ unlabelled B-Rep files from Onshape.	Massive scale; robust topology for pre-training.	No semantic labels; contains "garbage" data.

Table 3.1: Comparison of CAD Dataset Sources.

Based on the analysis detailed in Table 3.1, this paper will utilize a combination of CAD designs contained in Text2CAD as well as other online sources such as the ABC dataset and TraceParts. This combination ensures a balance between labeled semantic data (Text2CAD) and high-fidelity geometric variety (TraceParts). To help minimize discrepancies between annotations of several different datasets, a standard for annotation format mimicking the existing annotations in the Text2CAD dataset will be used.

3.1.2 Dataset Preprocessing

3.1.2.1 Dataset Filtering

Before training, the dataset undergoes a rigorous analysis to eliminate rare or low-quality CAD designs that could destabilize the model. The filtering process is divided into two distinct stages. The first stage is **Class and Format Standardization**, implemented using PythonOCC.

The script scans the raw data to enforce domain relevance by discarding any files not listed in the target object whitelist (e.g., excluding rare CAD designs and broken/corrupted files). Valid JSON-based CAD files are then automatically converted to STEP format using the OpenCascade library, while other unrecognizable formats are purged from the dataset.

The next stage focuses on **Manifold Validation**. Since raw STEP files frequently contain disjointed surfaces or unstitched faces, a "water-tight" topological check is enforced, Using volumetric analysis (Volume > 0), any file containing open shells or non-manifold geometry is explicitly discarded. This initial geometric sanitation prevents invalid outliers from polluting the training signal, ensuring that the model only learns from physically realizable 3D objects.

The second stage addresses statistical robustness by identifying and pruning *long-tail* categories, or more specifically, classes containing fewer than 100 unique geometric instances. This threshold is critical for preventing model overfitting. Deep learning models often resort to memorization rather than generalization when handling classes with insufficient sample sizes (Goodfellow et al., 2016). As demonstrated by C. Zhang et al. (2025), neural networks possess the capacity to fit arbitrary noise; in the absence of large-scale data constraints, they tend to memorize specific training instances rather than extracting abstract rules. This phenomenon leads to *shortcut learning*, where the model identifies objects via spurious noise patterns or texture statistics instead of meaningful topological features (Geirhos et al., 2020; Arpit et al., 2017).

Furthermore, recent studies indicate that this vulnerability is exacerbated in high-variability datasets. Research by You et al. (2025) demonstrates that machine learning models struggle significantly to identify low-occurrence classes where the "class pattern" breaks due to high intra-class variance. By enforcing a minimum threshold of 100 instances, we ensure that every semantic category (e.g., `GEAR`, `BRACKET`) possesses sufficient geometric variance to force the Transformer to learn a generalized representation of the object type, rather than memorizing a handful of outliers.

3.1.2.2 Dataset Standardization and Annotation Format

The next preprocessing step involves designing an annotation standard for the unannotated CAD dataset sources to ensure homogeneity with the pre-annotated Text2CAD dataset through a multi-stage standardization and formatting pipeline. The pipeline standardizes varied input structures into a consistent, unified representation matching the multi-level description schema of the Text2CAD dataset.

Data sourced from TraceParts often contains descriptive but non-standardized filenames (e.g., *"Hexagon_Head_Screw_ISO4017_M8.stp"*). To extract

the functional class, a **Hierarchical Regex Parser** is employed. The parser utilizes a dictionary of domain-specific keywords to map CAD files into general, high-level categories to ensure accurate annotation. The mapping logic follows a precedence order specified below to resolve ambiguities:

1. **Standard Extraction:** Strings containing ISO/DIN codes (e.g., "ISO 4017") are mapped immediately to their corresponding standard object family (e.g., `FASTENER`). This is done to exploit the rigorous standardization inherent in industrial engineering; parts explicitly labeled with international standards (ISO, DIN, ANSI) possess a verified, unambiguous functional identity, allowing them to serve as "Ground Truth" anchors for the classifier with near-100% label confidence.
2. **Morphological Keyword Search:** If no standard code is found, the parser scans for morphological roots. For example, filenames containing "gear", "pinion", or "worm" are all mapped to the class `GEAR` to simplify the manual annotation step.
3. **Exclusion Logic:** To prevent misclassification of assemblies, filenames containing "Assy" or "Assembly" are flagged and excluded from the single-part classification dataset, as assembly CAD files s inherently represent composite systems containing multiple distinct components (e.g., a gearbox comprising shafts, bearings, and housings). Including these files would introduce label ambiguity, as a single input graph would correspond to multiple conflicting functional classes, which is outside the research's scope.

Finally, while the hierarchical schema structure is retained for compatibility, the semantic objective of the annotation fields is redefined. Instead of describing the steps required to generate the object, the multi-level schema is used to categorize annotations of the same object based on their level of detail. Table 3.2 below shows the defined annotation levels with an example for each level.

Level	Scope	Definition	Example Annotation
Level 1 (Abstract)	**Functional Category**	Identifies semantic class and primary application. Useful for high-level retrieval.	"A standard metallic **fastener** designed for high-torque applications."
Level 2 (Concrete)	**Morphological Description**	Describes visual geometry and topological features (shape types, head styles) without specific dimensions.	"A **hexagonal-head bolt** featuring a partially threaded cylindrical shaft and a chamfered tip, with no washer face."
Level 3 (Detailed)	**Parametric Specification**	Provides precise constraints, dimensions, and standard designations required for reconstruction.	"An **ISO 4014 Hex Bolt** with a nominal thread diameter of **8mm** (M8), a total shaft length of **40mm**, and a thread pitch of 1.25mm."

Table 3.2: Proposed Hierarchical Annotation Levels for Parametric Captioning

3.1.2.3 Annotation Validation using BERTScore and Inter-Rater Reliability

Since the dataset is comprised of multiple sources, with most lacking the necessary descriptive texts, an additional annotation step is needed. Unlike simple classification, generating natural language descriptions requires high-fidelity textual data that automated processes cannot fully synthesize. To extend the dataset beyond the pre-annotated Text2CAD samples, a **Human-in-the-Loop (HITL)** annotation pipeline is employed. Unlike simple filtering, this process utilizes **BERTScore** (T. Zhang et al., 2020) as a semantic validation metric to guide the manual annotation process, ensuring that new descriptions align stylistically with the established Text2CAD annotation schema specified in Table 3.2.

The initial step includes writing manual annotations for each unannotated dataset entries which are validated by domain experts. The captions are designed to accurately describe the unlabelled geometric files (e.g., from TraceParts) with varying levels of detail. To maintain consistency, annotators follow a strict syntactic template derived from the Text2CAD dataset e.g. *"[Functional Component] with [Geometric Features] and [Dimensions]."* This ensures that the training data remains uniform across different sources.

To validate the quality of the human-generated descriptions, a blind cross-verification study was conducted. A senior domain expert will be assigned to review a random sample of $N = 500$ annotated descriptions, rating them as either "Compliant" or "Non-Compliant" based on the style

guide. The consistency of this quality control process was measured using **Inter-Rater Reliability (IRR)**, specifically **Cohen's Kappa (κ)** (Cohen, 1960). This metric was chosen over simple percent agreement to robustly account for the possibility of chance agreement, with a target threshold of $\kappa > 0.80$ established to certify the dataset's reliability for training the transformer model (Landis and G. G. Koch, 1977).

3.2 Model Design

This research proposes a novel *Hierarchical Dual-Stream Tokenizer* designed specifically to address the non-Euclidean nature of B-Rep (Boundary Representation) data. Unlike standard Natural Language Processing (NLP) tokenizers that treat input as a continuous 1D string which relies on the assumption that positional proximity equals semantic relationship, the proposed method treats the STEP file as a **topological graph**. This distinction is critical as in raw B-Rep data, geometric entities that are physically adjacent (e.g., two faces sharing an edge) may be defined thousands of lines apart in the ASCII file structure. Therefore, the data must be **normalized** to remove coordinate invariance, **compressed** to eliminate redundant entity definitions, and **linearized** via a deterministic graph traversal to preserve local geometric neighborhoods before ingestion.

3.2.1 Tokenizer Architecture

The tokenization pipeline consists of four distinct stages designed to ensure geometric invariance and computational efficiency. The overarching goal of the tokenizer is to bridge the *semantic gap* between the low-level mathematical definitions of CAD surfaces and the high-level semantic reasoning required by the Transformer. However, a standard tokenizer faces several issues when parsing STEP files. One such issue is *semantic fragmentation* of high-precision floating-point coordinates into meaningless sub-word units (e.g., splitting "10.125" into separate tokens "10", ".", "12", "5"), which destroys the numerical magnitude required for geometric reasoning (Wu et al., 2021). Second, standard sequential processing fails to capture the *non-local graph topology* inherent in B-Rep data; in a raw STEP file, spatially adjacent geometric features are often defined thousands of lines apart and linked only by arbitrary reference IDs (e.g., #243), creating long-range dependencies that exhaust the finite attention span of standard Transformer models (Wu et al., 2021). The proposed architecture aims to solve these limitations by integrating **Complex-Valued Embeddings** (Trabelsi et al., 2017) and **Rotary Positional Encodings** (Su et al., 2023). This allows the model to inherently encode geometric rotation and magnitude within the latent space, preserving topological fidelity without relying on reconstruction history. Figure 3.2 below shows an overview of the tokenizer architecture.

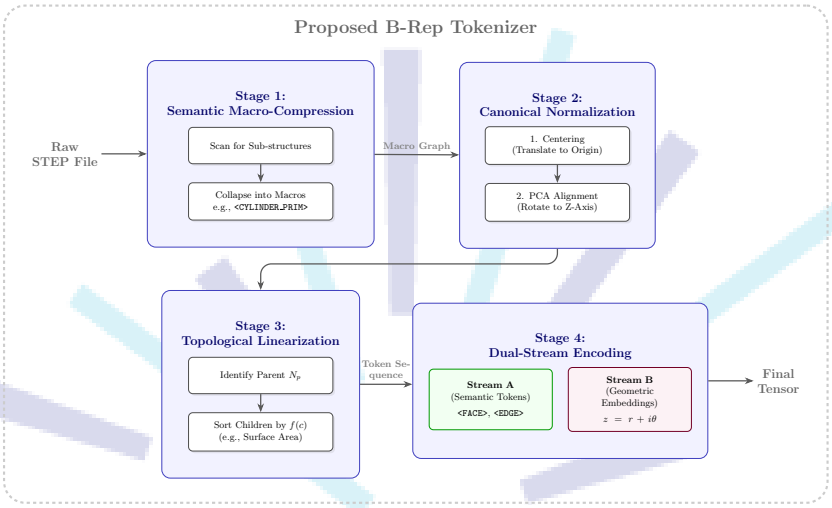

Figure 3.2: The proposed four-stage tokenization pipeline.

3.2.1.1 Stage 1: Semantic Macro-Compression

Raw STEP files exhibit extreme verbosity, where semantically simple manufacturing features are explicitly defined by dozens of constituent low-level topological entities. One example of this is objects such as a "Through-Hole" is represented not as a feature, but as a `CYLINDRICAL_SURFACE` connected to two `CIRCLE` edges, which are further linked to `VERTEX_POINT` definitions. As noted in foundational generative studies, such excessive sequence length dilutes the attention mechanism's ability to model global structure, often leading to convergence failure on complex assemblies (Wu et al., 2021). Furthermore, flat B-Rep graphs fail to capture the hierarchical nature of mechanical assemblies, necessitating a multi-level graph representation to efficiently encode topological adjacency (Colligan et al., 2022).

To mitigate this, a **Semantic Graph Contraction** algorithm is applied as a pre-tokenization step. A rule-based parser performs subgraph isomorphism checks to identify and compress known topological patterns into smaller, niche tokens. For instance, a `CYLINDRICAL_SURFACE` bounded by two `CIRCLE` edges is fused into a single macro-node `<CYLINDER_PRIMITIVE>`. Similarly, adjacent chains of tangential `B_SPLINE_SURFACE` patches are aggregated into `<FILLET_CHAIN>` macros. This heuristic reduces the generated sequence length while retaining the semantic essence of the geometry, effectively increasing the model's receptive field and allowing the Transformer to reason about feature-level relationships rather than low-level surface connectivity.

3.2.1.2 Stage 2: Canonical Geometric Normalization

Standard STEP coordinates are sensitive to global positioning, i.e. a single mechanical component defined at coordinate $(0,0,0)$ might generate different token sequences than the same component translated to $(100,100,100)$. A sufficiently accurate model must achieve translational and rotational invariance, meaning that its output is not dependent on the translation and rotation of the input data (J. Li et al., 2023; Muaz, 2021). To achieve this, the B-Rep graph undergoes a rigorous **Canonical Alignment** process. This reduces the dimensionality of the learning manifold by ensuring that all semantically identical objects occupy the same region in the input vector space (Wu et al., 2021). The normalization pipeline is implemented using three mathematical transformations described below:

1. **Centroid Subtraction (Translation Invariance):** The geometric center of mass μ is computed across all control points $P = \{p_1, p_2, ..., p_N\}$ in the solid. All vertices are then translated such that the centroid aligns with the global origin:

$$p'_i = p_i - \frac{1}{N}\sum_{j=1}^{N} p_j \tag{3.1}$$

2. **PCA Alignment (Rotation Invariance):** To align the geometry with a consistent canonical orientation, Principal Component Analysis (PCA) is performed on the vertex cloud (Chaouch and Verroust-Blondet, 2009). We compute the covariance matrix Σ:

$$\Sigma = \frac{1}{N}\sum_{i=1}^{N} p'_i (p'_i)^T \tag{3.2}$$

   The eigenvectors v_1, v_2, v_3 of Σ (corresponding to the largest to smallest eigenvalues) represent the primary, secondary, and tertiary axes of variance. The entire geometry is rotated via a matrix $R = [v_1, v_2, v_3]$ such that the axis of greatest variance (e.g., the length of a screw) aligns with the global z-axis.

3. **Ambiguity Resolution:** A standard PCA alignment suffers from sign ambiguity (the "180-degree flip" problem), where a bolt might be aligned pointing Up or Down randomly. To resolve this, a deterministic "heavy-bottom" rule is enforced: if the density of points in the negative z half-space is lower than the positive half-space, the axis is flipped ($z' = -z$). This ensures consistent orientation across the dataset (Vranic, 2003).

3.2.1.3 Stage 3: Topological Linearization (Canonical DFS)

Since Transformers utilize positional encoding to interpret sequence order, the serialization of the B-Rep graph must be strictly deterministic. A standard Depth-First Search (DFS) is insufficient because it is sensitive

to the stochastic permutation of entity identifiers in the STEP file (e.g., visiting `#10=FACE` before `#20=FACE` purely based on arbitrary ID assignment). This introduces **Graph Isomorphism Ambiguity**: two geometrically identical cubes could produce entirely different token sequences if their internal numbering differs, forcing the model to waste capacity learning to ignore these permutations (Wu et al., 2021).

Resolving the ambiguity requires the implementation of a **Canonical DFS** traversal. The algorithm introduces a strict ordering constraint at every branch of the graph, where for any parent node N_p (e.g., a `SHELL`) with a set of child nodes $C = \{c_1, c_2, ..., c_k\}$ (e.g., `FACES`), the children are strictly sorted based on a geometric invariant function vector $\mathbf{f}(c)$ before the traversal continues. The ordering logic is defined as:

$$\text{Order}(C) = \text{sort}_{\downarrow}\left(\{\mathbf{f}(c) \mid c \in C\}\right) \tag{3.3}$$

In this research, the invariant function $\mathbf{f}(c)$ is designed to be invariant to both rigid body rotation and file re-indexing by utilizing a **Hierarchical Sorting Key** which consists of 3 parts:

- **Primary Key (k_1):** *Geometric Magnitude*. For Topological Faces, this is the **Surface Area**; for Edges, it is the **Curve Length**. This ensures that large, dominant features (like the main body of a housing) appear early in the sequence, providing a stable "context" for the attention mechanism.
- **Secondary Key (k_2):** *Centroid Distance*. To resolve symmetries (tie-breaking), we calculate the Euclidean distance from the entity's centroid to the global origin $(0,0,0)$.
- **Tertiary Key (k_3):** *Topological Degree*. The number of connected edges/loops.

3.2.1.3.1 Handling Symmetries (Tie-Breaking) In cases of perfect symmetry (e.g., a Cube where all 6 faces have identical Area and Centroid Distance), a deterministic fallback is required. The fallback procedure utilizes the principal axis alignment from Stage 2: faces are sorted by their centroid's Z, Y, X coordinates respectively. This guarantees that a specific 3D geometry yields an *identical* token sequence regardless of the permutation of entity IDs in the raw file (Colligan et al., 2022).

3.2.1.4 Stage 4: Complex-Valued Dual-Stream Encoding

To preserve high-fidelity precision (e.g., NURBS weights) without bloating the discrete vocabulary, a Dual-Stream architecture is used. The tokenizer emits two parallel streams for every step t:

- **Stream A (Semantic Tokens):** Discrete integers representing the categorical type of entity (e.g., `<FACE>`, `<NURBS_CURVE>`).
- **Stream B (Geometric Embeddings):** A 128-dimensional **Complex-Valued Vector** encoding the shape parameters.

The complex vector $v \in \mathbb{C}^{64}$ encodes the geometry as $z = r + i\theta$. The real part r represents scale-invariant magnitudes (e.g., radius, edge length), while the imaginary part θ encodes the relative orientation. This formulation is inspired by Euler's formula ($e^{i\theta}$), allowing the model to perform rotation operations purely through phase shifts in the complex domain.

3.2.2 Model Parameters

The core model is a Transformer Encoder based on the BERT architecture (Devlin et al., 2019), modified to accept the Dual-Stream input.

- **Embedding Dimension (d_{model}):** 512. The discrete tokens are embedded into $\mathbb{R}^{512}$, and the 128-dim complex geometric vectors are projected via a Multi-Layer Perceptron (MLP) to $\mathbb{R}^{512}$ before summation.
- **Attention Heads:** 8 Parallel heads.
- **Layers:** 12 Encoder blocks.
- **Context Window:** 2048 tokens.
- **Feed-Forward Network:** GELU activation with a dimension of 2048.

The selection of these hyperparameters is governed by the specific trade-offs inherent to B-Rep graph learning. While modern LLMs often utilize dimensions upwards of 4096, CAD syntax has a significantly smaller vocabulary size (< 1000 unique structural tokens). A dimension of $d_{model} = 512$ was chosen to prevent overfitting on this sparse vocabulary while providing sufficient manifold capacity to represent the fused Dual-Stream input.

The choice of 12 layers aligns with the hierarchical nature of boundary representation. Lower layers (L_{1-4}) are tasked with learning local topological syntax (e.g., edge continuity), intermediate layers (L_{5-8}) aggregate these into feature-level representations (e.g., pockets, bosses), and upper layers (L_{9-12}) extract high-level semantic class information.

Standard BERT implementations utilize a 512-token window. However, B-Rep graphs—even after the proposed Macro-Compression—exhibit long-range dependencies where a face defined at token T_1 may reference a surface defined at token T_{1000}. A window of 2048 tokens was empirically selected to encompass the complete topological graph of 95% of the dataset samples without necessitating destructive truncation.

3.2.2.1 Geometry-Infused Attention Mechanism

Unlike standard NLP Transformers where the attention score is derived solely from semantic word embeddings, the proposed architecture employs a **Geometry-Infused Attention** mechanism. This mechanism is

responsible for synthesizing the Dual-Stream output from the tokenizer into a unified representation.

The input to the attention layer for the i-th token is defined as the fusion of the semantic and geometric streams:

$$X_i = \text{Embed}(T_i) + \text{MLP}(V_i) \tag{3.4}$$

Where T_i is the discrete semantic token (e.g., `<FACE>`) and V_i is the 128-dimensional complex geometric vector. The Multi-Layer Perceptron (MLP) acts as a bridge, projecting the continuous geometric manifold into the semantic model dimension d_{model}.

3.2.2.1.1 Geometric Rotary Positional Integration To strictly enforce the rotational invariance defined in the methodology, the model utilizes a modified version of Rotary Positional Embeddings (RoPE). The "Phase" component (θ) of the complex geometric input V_i is directly mapped to the rotation of the Query (Q) and Key (K) vectors in the attention calculation:

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{(R_{\theta_i}Q)(R_{\theta_j}K)^T}{\sqrt{d_k}}\right)V \tag{3.5}$$

Here, R_θ represents the rotation matrix derived from the tokenizer's complex input. This ensures that the attention score between two CAD entities depends not only on their semantic type but also on their **relative geometric orientation**. For example, a `SCREW` entity will attend strongly to a `HOLE` entity only if their orientation vectors (Normal vectors) are aligned (i.e., the relative rotation $\Delta\theta \approx 0$), thereby embedding physical assembly logic directly into the attention weights.

3.2.2.2 Classification Head

The final representation is extracted via Global Average Pooling across the sequence length, condensing the topological graph into a single latent vector $Z_{graph} \in \mathbb{R}^{512}$. This vector is passed to a softmax classifier to predict the functional category of the mechanical part, adhering to the class schema defined in the data gathering phase.

3.3 Training and Backtesting

3.3.1 Training Setup

The classification objective is optimized using **Categorical Cross-Entropy Loss**. This function penalizes the divergence between the predicted probability distribution $\hat{y}$ and the ground truth one-hot vector y. For a dataset of N samples and C classes, the loss $\mathscr{L}$ is defined as:

$$\mathscr{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c}) \tag{3.6}$$

Where $\hat{y}_{i,c}$ is the softmax probability output of the model for class c. The logarithmic nature of the loss function imposes a heavy penalty on confident but incorrect predictions, which is critical for distinguishing between visually similar CAD classes (e.g., distinguishing an ISO-4014 Bolt from an ISO-4017 Bolt based on subtle shaft length variations).

3.3.2 Evaluation Metrics

To validate the effectiveness of the proposed tokenizer, the model is evaluated on two fronts:

1. **Standard Classification Metrics:** Accuracy, Precision, Recall, and F1-Score on a held-out test set (20% of the data).
2. **Rotational Robustness Test:** A specialized test set is generated by applying random random $SO(3)$ rotations to the test data. The model's accuracy on this rotated set is compared to the baseline to verify the efficacy of the Canonical Alignment and Complex-Valued Embeddings.

REFERENCES

[1] A. Aranburu, D. Justel, and I. Angulo, "Reusability and flexibility in parametric surface-based models: A review of modelling strategies," *Computer-Aided Design and Applications*, vol. 18, pp. 864–874, Apr. 2021. DOI: 10.14733/cadaps.2021.864-874.

[2] D. Arpit et al., "A closer look at memorization in deep networks," *arXiv (Cornell University)*, Jan. 2017. DOI: 10.48550/arxiv.1706.05394.

[3] D. S. Asudani, N. K. Nagwani, and P. Singh, "Impact of word embedding models on text analytics in deep learning environment: A review," *Artificial Intelligence Review*, vol. 56, pp. 1–81, Feb. 2023. DOI: 10.1007/s10462-023-10419-1.

[4] D. L. Banks and S. E. Fienberg, *Data Mining, Statistics*, Third Edition, R. A. Meyers, Ed. New York: Academic Press, 2003, pp. 247–261, ISBN: 978-0-12-227410-7. DOI: https://doi.org/10.1016/B0-12-227410-5/00164-2. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B0122274105001642.

[5] M. Blessing, "Ai-driven data classification and organization," Sep. 2021.

[6] S. Borrohou, R. Fissoune, and H. Badir, "The role of data transformation in modern analytics: A comprehensive survey," *Journal of Computer Languages*, vol. 84, p. 101 329, 2025, ISSN: 2590-1184. DOI: https://doi.org/10.1016/j.cola.2025.101329. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2590118425000152.

[7] M. Chaouch and A. Verroust-Blondet, "Alignment of 3d models," *Graphical Models*, vol. 71, no. 2, pp. 63–76, 2009.

[8] K. Chitturi, "Understanding vector embeddings in ai search: A technical deep dive," *INTERNATIONAL JOURNAL OF COMPUTER ENGINEERING & TECHNOLOGY*, vol. 15, pp. 1725–1733, Dec. 2024. DOI: 10.34218/IJCET_15_06_147.

[9] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, pp. 37–46, 1960. DOI: 10.1177/001316446002000104.

[10] A. Colligan, T. Robinson, D. Nolan, Y. Hua, and W. Cao, "Hierarchical cadnet: Learning from b-reps for machining feature recognition," *Computer-Aided Design*, vol. 147, p. 103 226, 2022. DOI: 10.1016/j.cad.2022.103226.

[11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds., Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423. [Online]. Available: https://aclanthology.org/N19-1423/.

[12] D. Fleche, J.-B. Bluntzer, M. Mahdjoub, and J.-C. Sagot, "First step toward a quantitative approach to evaluate collaborative tools," in Dec. 2014, vol. 21, pp. 288–293. DOI: 10.1016/j.procir.2014.03.160.

[13] R. Geirhos et al., "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, pp. 665–673, Nov. 2020. DOI: 10.1038/s42256-020-00257-z.

[14] M. Geva, R. Schuster, J. Berant, and O. Levy, "Transformer feed-forward layers are key-value memories," *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, Eds., pp. 5484–5495, Jan. 2021. DOI: 10.18653/v1/2021.emnlp-main.446. Accessed: Oct. 10, 2023. [Online]. Available: https://aclanthology.org/2021.emnlp-main.446/.

[15] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT Press, 2016.

[16] C. Gümeli, A. Dai, and M. Nießner, *Roca: Robust cad model retrieval and alignment from a single image*, arXiv.org, Jun. 2022. DOI: 10.1109/CVPR52688.2022.00399. Accessed: May 13, 2024. [Online]. Available: https://arxiv.org/abs/2112.01988.

[17] S. Gutiérrez de Ravé, E. Gutiérrez de Ravé, and F. J. Jiménez-Hornero, "Enhancing efficiency and creativity in mechanical drafting: A comparative study of general-purpose cad versus specialized toolsets," *Applied System Innovation*, vol. 8, pp. 74–74, May 2025. DOI: 10.3390/asi8030074. Accessed: Jun. 12, 2025. [Online]. Available: https://www.mdpi.com/2571-5577/8/3/74.

[18] L. Hackman, P. Mack, and H. Ménard, "Behind every good research there are data. what are they and their importance to forensic science," *Forensic Science International: Synergy*, pp. 100 456–100 456, Feb. 2024. DOI: 10.1016/j.fsisyn.2024.100456.

[19] J. Herschel, *On the Action of the Rays of the Solar Spectrum on Vegetable Colours, and on Some New Photographic Processes* (On the Action of the Rays of the Solar Spectrum on Vegetable Colours and on Some New Photographic Processes). Taylor, 1842. [Online]. Available: https://books.google.co.id/books?id=hsBHV4j9QWwC.

[20] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, pp. 1735–1780, Nov. 1997. DOI: 10.1162/neco.1997.9.8.1735. [Online]. Available: https://direct.mit.edu/neco/article-abstract/9/8/1735/6109/Long-Short-Term-Memory?redirectedFrom=fulltext.

[21] C. Y. Ip and W. C. Regli, "Automatic classification of cad models," Apr. 2005. DOI: 10.17918/etd-505. Accessed: Oct. 25, 2025.

[22] A. Kanezaki, Y. Matsushita, and Y. Nishida, "Rotationnet: Joint object categorization and pose estimation using multiviews from unsupervised viewpoints," *arXiv:1603.06208 [cs]*, Mar. 2018. Accessed: Nov. 13, 2021. [Online]. Available: https://arxiv.org/abs/1603.06208.

[23] W. Karl et al., "Fusion 360 gallery: A dataset and environment for programmatic cad construction from human design sequences," *arXiv.org*, May 2021. DOI: https://doi.org/10.48550/arXiv.2010.02392. Accessed: Dec. 8, 2025. [Online]. Available: https://arxiv.org/abs/2010.02392v2.

[24] M. S. Khan, S. Sinha, T. U. Sheikh, D. Stricker, S. A. Ali, and M. Z. Afzal, "Text2cad: Generating sequential cad models from beginner-to-expert level text prompts," *arXiv.org*, Sep. 2024. DOI: https://doi.org/10.48550/arXiv.2409.17106. Accessed: Dec. 8, 2025. [Online]. Available: https://arxiv.org/abs/2409.17106v1.

[25] S. Koch et al., "Abc: A big cad model dataset for geometric deep learning," *arXiv.org*, Apr. 2019. DOI: https://doi.org/10.48550/arXiv.1812.06216. Accessed: Dec. 8, 2025. [Online]. Available: https://arxiv.org/abs/1812.06216v2.

[26] L. Kozma and J. Voderholzer, *Theoretical analysis of byte-pair encoding*, 2024. arXiv: 2411.08671 [cs.DS]. [Online]. Available: https://arxiv.org/abs/2411.08671.

[27] S. Kukreja, T. Kumar, V. Bharate, A. Purohit, A. Dasgupta, and D. Guha, "Vector databases and vector embeddings-review," pp. 231–236, 2023. DOI: 10.1109/IWAIIP58158.2023.10462847.

[28] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977. Accessed: Dec. 2025. [Online]. Available: http://www.jstor.org/stable/2529310.

[29] A. Lane and J. B. Adelusi, *Role of data classification in enhancing security in big data environments*, ResearchGate, Jul. 2023. [Online]. Available: https://www.researchgate.net/publication/388067206_Role_of_Data_Classification_in_Enhancing_Security_in_Big_Data_Environments.

[30] J. Li, J. Chen, Y. Tang, C. Wang, B. A. Landman, and S. K. Zhou, "Transforming medical imaging with transformers? a comparative review of key properties, current progresses, and future perspectives," *Medical Image Analysis*, vol. 85, p. 102 762, 2023. DOI: https://doi.org/10.1016/j.media.2023.102762. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1361841523000233.

[31] M. Li, Y. Zhang, J. Fuh, and Z. Qiu, "Design reusability assessment for effective cad model retrieval and reuse," *International Journal of Computer Applications in Technology, 2011*, vol. 40, pp. 3–12, Feb. 2011. DOI: 10.1504/IJCAT.2011.038546. Accessed: Oct. 13, 2025. [Online]. Available: https://www.inderscience.com/info/inarticle.php?artid=38546.

[32] Y. Liu, G. Tilahun, X. Gao, Q. Wen, and M. Gervers, *A comparative study of static and contextual embeddings for analyzing semantic changes in medieval latin charters*, 2025. Accessed: Oct. 12, 2025. [Online]. Available: https://aclanthology.org/2025.loreslm-1.14.pdf.

[33] D. Machalica and M. Matyjewski, "Cad models clustering with machine learning," *Archive of Mechanical Engineering*, vol. 66, no. 2, pp. 133–152, Apr. 2019. DOI: 10.24425/ame.2019.128441. Accessed: Oct. 25, 2025. [Online]. Available: https://yadda.icm.edu.pl/baztech/element/bwmeta1.element.baztech-77666cdd-ada0-472e-b608-86fafaa40164.

[34] L. Mandelli and S. Berretti, *Cad 3d model classification by graph neural networks: A new approach based on step format*, arXiv.org, Oct. 2022. Accessed: Oct. 13, 2025. [Online]. Available: https://arxiv.org/abs/2210.16815.

[35] L. Mason, "Data vs information," *www.academia.edu*, Jan. 2019. [Online]. Available: https://www.academia.edu/39505535/Data_vs_Information.

[36] D. Maturana and S. Scherer, "Voxnet: A 3d convolutional neural network for real-time object recognition," in *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2015, pp. 922–928. DOI: 10.1109/IROS.2015.7353481.

[37] J. D. McGlothlin, "Health hazard evaluation report: Hhe-80-248-791, national oceanic and atmospheric administration, u.s. dept of commerce, washington, d.c.," *Health Hazard Evaluation Reports (HHE)*, pp. 1–10, Jan. 1981. DOI: 10.26616/nioshhhe80248791. Accessed: Nov. 28, 2025. [Online]. Available: https://www.cdc.gov/niosh/hhe/reports/pdfs/80-248-791.pdf.

[38] M. El-Mehalawi and R. Allen Miller, "A database system of mechanical components based on geometric and topological similarity. part i: Representation," *Computer-Aided Design*, vol. 35, pp. 83–94, Jan. 2003. DOI: 10.1016/s0010-4485(01)00177-4. Accessed: Apr. 25, 2022.

[39] M. El-Mehalawi and R. Allen Miller, "A database system of mechanical components based on geometric and topological similarity. part ii: Indexing, retrieval, matching, and similarity assessment," *Computer-Aided Design*, vol. 35, pp. 95–105, Jan. 2003. DOI: 10.1016/s0010-4485(01)00178-6. Accessed: Apr. 25, 2022.

[40] U. Muaz, *Invariance, causality, and robust deep learning*, Medium, May 2021. Accessed: Dec. 9, 2025. [Online]. Available: https://medium.com/data-science/invariance-causality-and-robust-deeplearning-df8db9091627.

[41] W. Newhouse, M. Souppaya, J. Kent, K. Sandlin, and K. Scarfone, "Data classification concepts and considerations for improving data protection," *Data Classification Concepts and Considerations for Improving Data Protection*, Nov. 2023. DOI: 10.6028/nist.ir.8496.ipd. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/ir/2023/NIST.IR.8496.ipd.pdf.

[42] H.-N. Nguyen and L. Cuong, "Overview of data classification and applications in data security," *International Journal of Environmental Sciences*, vol. 11, pp. 31–39, Jun. 2025. DOI: 10.64252/ac0b3685. Accessed: Jul. 20, 2025.

[43] J. O'Donnell and C. Crownhart, *We did the math on ai's energy footprint. here's the story you haven't heard.* MIT Technology Review, May 2025. Accessed: Nov. 28, 2025. [Online]. Available: https://www.technologyreview.com/2025/05/20/1116327/ai-energy-usage-climate-footprint-big-tech/.

[44] K. Olson, "What are data?" *Qualitative Health Research*, vol. 31, no. 8, 1015960, 2021. DOI: 10.1177/10497323211015960.

[45] O. Omolayo and O. Babatope, "Comparative evaluation of vector embedding frameworks for scalable sematic retrieval in pdf-based rag systems," *International Journal of Research Publication and Reviews*, vol. 6, pp. 12 506–12 511, Jun. 2025. DOI: 10.55248/gengpi.6.0625.23100.

[46] P. Pajo, "Vector embeddings unveiled: A comprehensive exploration of their creation, types, applications,...," *ResearchGate*, Apr. 2025. DOI: 10.13140/RG.2.2.15544.05129. [Online]. Available: https://www.researchgate.net/publication/390598346_Vector_Embeddings_Unveiled_A_Comprehensive_Exploration_of_Their_Creation_Types_Applications_Challenges_and_Future_Directions_in_Machine_Learning/.

[47] P. Papadakis, I. Pratikakis, T. Theoharis, and S. Perantonis, "Panorama: A 3d shape descriptor based on panoramic views for unsupervised 3d object retrieval," *International Journal of Computer Vision*, vol. 89, pp. 177–192, Aug. 2009. DOI: 10.1007/s11263-009-0281-6. Accessed: Dec. 7, 2022.

[48] F. Parmeggiani, "Fear of the dark: Diazo printing by photochemical decomposition of aryldiazonium tetrafluoroborates," *Journal of Chemical Education*, vol. 91, pp. 692–695, Feb. 2014. DOI: 10.1021/ed400555a. Accessed: Jul. 24, 2025.

[49] R. Patil, S. Boit, V. Gudivada, and J. Nandigam, "A survey of text representation and embedding techniques in nlp," *IEEE Access*, vol. 11, pp. 36 120–36 146, 2023. DOI: 10.1109/access.2023.3266377.

[50] M. J. Pratt, "Introduction to iso 10303—the step standard for product data exchange," *Journal of Computing and Information Science in Engineering*, vol. 1, pp. 102–103, Jan. 2001. DOI: 10.1115/1.1354995. Accessed: May 4, 2022.

[51] W. Qader, M. M. Ameen, and B. Ahmed, "An overview of bag of words;importance, implementation, applications, and challenges," pp. 200–204, Jun. 2019. DOI: 10.1109/IEC47844.2019.8950616.

[52] S. Qaiser and R. Ali, "Text mining: Use of tf-idf to examine the relevance of words to documents," *International Journal of Computer Applications*, vol. 181, Jul. 2018. DOI: 10.5120/ijca2018917395.

[53] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, *Pointnet: Deep learning on point sets for 3d classification and segmentation*, arXiv.org, 2016. [Online]. Available: https://arxiv.org/abs/1612.00593.

[54] M. Rahman, M. Khatib, P. Rani, S. Shaikh, and G. Gunashekar, "Effect of computer aided drafting on manual drafting skills," *International Journal of Innovative Technology and Exploring Engineering*, vol. 8, pp. 3012–3014, Sep. 2019. DOI: 10.35940/ijitee.K2315.0981119.

[55] R. Roj, M. Sommer, H.-B. Woyand, R. Theiß, and P. Dueltgen, "Classification of cad-models based on graph structures and machine learning," *Computer-Aided Design and Applications*, vol. 19, pp. 449–469, Sep. 2021. DOI: 10.14733/cadaps.2022.449-469.

[56] I. Sadrehaghighi, "Computer aided design (cad)," Jan. 2022. DOI: 10.13140/RG.2.2.12634.62408.

[57] A. I. J. I. B. W. Samuels, "One-hot encoding and two-hot encoding: An introduction," Jan. 2024. DOI: 10.13140/RG.2.2.21459.76327.

[58] J. Schoonenboom, "The fundamental difference between qualitative and quantitative data in mixed methods research," *Forum Qualitative Sozialforschung / Forum: Qualitative Social Research*, vol. 24, Jan. 2023. DOI: http://dx.doi.org/10.17169/fqs-24.1.3986. Accessed: Oct. 13, 2025. [Online]. Available: https://www.qualitative-research.net/index.php/fqs/article/view/3986/4916.

[59] K. Sfikas, I. Pratikakis, and T. Theoharis, "Ensemble of panorama-based convolutional neural networks for 3d model classification and retrieval," *Computers & Graphics*, vol. 71, pp. 208–218, 2018, ISSN: 0097-8493. DOI: https://doi.org/10.1016/j.cag.2017.12.001. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0097849317301978.

[60] K. Sfikas, T. Theoharis, and I. Pratikakis, "Exploiting the PANORAMA Representation for Convolutional Neural Network Classification and Retrieval," in *Eurographics Workshop on 3D Object Retrieval*, I. Pratikakis, F. Dupont, and M. Ovsjanikov, Eds., The Eurographics Association, 2017, ISBN: 978-3-03868-030-7. DOI: 10.2312/3dor.20171045.

[61] A. Sherstinsky, "Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network," *Physica D: Nonlinear Phenomena*, vol. 404, p. 132 306, Mar. 2020. DOI: 10.1016/j.physd.2019.132306.

[62] S. K. Shivayogi, "Data classification methodologies and implementation," *Journal of Computer Science and Technology Studies*, vol. 7, pp. 202–210, May 2025. DOI: 10.32996/jcsts.2025.7.5.26. Accessed: May 31, 2025.

[63] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, "Roformer: Enhanced transformer with rotary position embedding," *CoRR*, vol. abs/2104.09864, Nov. 2023. DOI: https://doi.org/10.48550/arXiv.2104.09864. [Online]. Available: https://arxiv.org/abs/2104.09864.

[64] I. E. Sutherland, "Sketch pad a man-machine graphical communication system," in *Proceedings of the SHARE design automation workshop*, 1964, pp. 6–329.

[65] B. Suyunu, E. Taylan, and A. Özgür, *Linguistic laws meet protein sequences: A comparative analysis of subword tokenization methods*, 2024. arXiv: 2411.17669 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2411.17669.

[66] M. Szilvsi-Nagy and G. Mátyási, "Analysis of stl files," *Mathematical and Computer Modelling*, vol. 38, Oct. 2003. DOI: 10.1016/S0895-7177(03)90079-3.

[67] T. Taipalus, "Vector database management systems: Fundamental concepts, use-cases, and current challenges," *Cognitive Systems Research*, vol. 85, p. 101 216, 2024, ISSN: 1389-0417. DOI: https://doi.org/10.1016/j.cogsys.2024.101216. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1389041724000093.

[68] P. Taylor, *Data growth worldwide 2010-2028*, Statista, Jun. 2025. Accessed: Oct. 12, 2025. [Online]. Available: https://www.statista.com/statistics/871513/worldwide-data-created/?srsltid=AfmBOoos3sZoZKw6aS7wf2CiXFwWvkUGV6VUzUBHutfpJE82JTtKg53o.

[69] C. Trabelsi et al., "Deep complex networks," *CoRR*, vol. abs/1705.09792, 2017. DOI: https://doi.org/10.48550/arXiv.1705.09792. Accessed: Dec. 8, 2025. [Online]. Available: https://arxiv.org/abs/1705.09792.

[70] H. Tran, *Natural Language Processing for Everyone*. Bookdown, 2024. [Online]. Available: https://bookdown.org/tranhungydhcm/mybook/.

[71] G. V. Trunk, "A problem of dimensionality: A simple example," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-1, pp. 306–307, Jul. 1979. DOI: 10.1109/TPAMI.1979.4766926. Accessed: Dec. 29, 2021. [Online]. Available: https://ieeexplore.ieee.org/document/4766926.

[72] A. Vaswani et al., "Attention is all you need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17, Long Beach, California, USA: Curran Associates Inc., 2017, pp. 6000–6010, ISBN: 9781510860964.

[73] E. Voita, D. Talbot, F. Moiseev, R. Sennrich, and I. Titov, *Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned*, 2019. Accessed: Oct. 12, 2025. [Online]. Available: https://aclanthology.org/P19-1580.pdf.

[74] D. Vranic, "3d model retrieval," in *PhD Thesis, University of Leipzig*, Section 3.2: Pose Normalization, 2003.

[75] B. Wang, A. Wang, F. Chen, Y. Wang, and C.-C. Kuo, "Evaluating word embedding models: Methods and experimental results," *APSIPA Transactions on Signal and Information Processing*, vol. 8, Jan. 2019. DOI: 10.1017/ATSIP.2019.12.

[76] R. Wu, C. Xiao, and C. Zheng, "Deepcad: A deep generative network for computer-aided design models," *arXiv.org*, Aug. 2021. DOI: https://doi.org/10.48550/arXiv.2105.09492. Accessed: Dec. 8, 2025. [Online]. Available: https://arxiv.org/abs/2105.09492.

[77] C. You, H. Dai, Y. Min, J. S. Sekhon, S. Joshi, and J. S. Duncan, "Uncovering memorization effect in the presence of spurious correlations," *Nature Communications*, vol. 16, Jul. 2025. DOI: 10.1038/s41467-025-61531-5. Accessed: Jul. 24, 2025. [Online]. Available: https://www.nature.com/articles/s41467-025-61531-5.

[78] I. Zeid, *Mastering Solidworks*. Macromedia Press, 2021.

[79] *Understanding deep learning requires rethinking generalization*, ICLR 2017, Oct. 2025. Accessed: Dec. 8, 2025. [Online]. Available: https://openreview.net/forum?id=Sy8gdB9xx.

[80] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "Bertscore: Evaluating text generation with bert," *CoRR*, vol. abs/1904.09675, Feb. 2020. DOI: https://doi.org/10.48550/arXiv.1904.09675. Accessed: Dec. 9, 2025. [Online]. Available: http://arxiv.org/abs/1904.09675.